

CP-Agent: Agentic Constraint Programming

Stefan Szeider
Faculty of Informatics
TU Wien
Wien, Austria
stefan@szeider.net

Abstract

The translation of natural language to formal constraint models requires expertise in the problem domain and modeling frameworks. To explore the effectiveness of agentic workflows, we propose CP-Agent, a Python coding agent that uses the ReAct framework with a persistent IPython kernel. We provide the relevant domain knowledge as a project prompt of under 50 lines. The algorithm works by iteratively executing code, observing the solver’s feedback, and refining constraint models based on execution results.

We evaluate CP-Agent on 101 constraint programming problems from CP-Bench. We made minor changes to the benchmark to address systematic ambiguities in the problem specifications and errors in the ground-truth models. On the clarified benchmark, CP-Agent achieves perfect accuracy on all 101 problems. Our experiments show that minimal guidance outperforms detailed procedural scaffolding. Our experiments also show that explicit task management tools can have both positive and negative effects on focused modeling tasks.

ACM Reference Format:

Stefan Szeider. 2026. CP-Agent: Agentic Constraint Programming. In *3rd International Workshop on Large Language Models For Code (LLM4Code ’26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3786181.3788711>

1 Introduction

Constraint Programming is a declarative paradigm for solving combinatorial problems in which users specify the constraints a solution must satisfy, rather than prescribing how to find it [26]. The main obstacle preventing wider adoption of CP remains the modeling bottleneck. Converting natural language problem descriptions into formal constraint models requires deep expertise in both the problem domain and the modeling framework [5]. This challenge is referred to as the “holy grail” of constraint programming [24].

In recent years, Large Language Models (LLMs) have been successfully applied to automate this translation process. We considered CP-Bench [14], a benchmark introduced to evaluate LLM-driven constraint modeling capabilities, which contains 101 diverse combinatorial problems sourced from various established CP community resources. These problems are used as benchmarks for optimization and satisfaction tasks, and include a variety of distinct constraint types. This benchmark addresses the limitations of existing datasets, such as NL4Opt [17] and Logic Grid Puzzles [25],

which lack the diversity and complexity needed to evaluate constraint modeling capabilities.

Michailidis et al. [14] evaluated several LLM-based workflows on CP-Bench. With direct prompting, Python-based frameworks (CPMpy, OR-Tools) achieve overall good accuracy up to 65%. Indeed, MiniZinc (a popular constraint modeling language) peaked at around 50%. Repeated sampling can be combined with self-verification to improve accuracy for Python-based frameworks to approximately 70%.

The relationship between fixed and agentic workflows is now well understood. Chain-of-thought prompting [29] and subsequent work [19, 32] have been widely used in the literature. One distinguishes between *fixed* workflows that use well-scoped, short-horizon tasks and *agentic* workflows where the LLM runs in a loop and uses tools. We hypothesize that constraint modeling from natural language falls into the category of tasks that require an agentic workflow with iterative refinement, hypothesis testing, and dynamic adaptation based on solver feedback. Our experiments show that this is indeed the case.

To test this hypothesis, we introduce *CP-Agent*, a constraint modeling agent that utilizes our general-purpose *agentic-python-coder* system. CP-Agent maintains a persistent *IPython* kernel [16], enabling stateful and iterative development. Instead of domain-specific logic embedded in its architecture, we use a concise project prompt as the guide. The agent can execute code, receive runtime feedback from the constraint solver, and adjust its constraint model accordingly.

The underlying coding agent is available as an *MCP server* [1]. The Model Context Protocol (MCP), now governed by the Linux Foundation, is widely used for connecting language models with external tools [8]. With this integration, the CP agent can be used from any MCP-compatible client, including coding assistants like Cursor or AI chatbots like Claude Desktop.

On the CP-Bench dataset of 101 problems (with added clarifications for some ambiguous problems and output format requirements), CP-Agent solves all problems correctly. We use Claude Sonnet 4.5 as the primary reference language model, though we also tested this with other models from different providers. Just by changing the prompt while keeping the agent architecture, we can adapt the system to different domains. This is exemplified by follow-up work [23] on modeling tasks for logic programming.

In addition, we provide an empirical evaluation of using a task management tool within our agentic coding workflow. The results are mixed: task management is helpful for some but counterproductive for others; the net effect was adverse, with significantly increased token usage, suggesting that the overhead can distract as often as it organizes.



This work is licensed under a Creative Commons Attribution 4.0 International License. *LLM4Code ’26, Rio de Janeiro, Brazil*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2412-1/26/04
<https://doi.org/10.1145/3786181.3788711>

2 Related Work

Our approach follows the neurosymbolic paradigm, combining neural AI with symbolic constraint satisfaction [18]. The LLM-based agent converts natural language into formal constraints. The CP solver guarantees formal correctness of the computation.

LLM agents have been explored for software engineering tasks. SWE-agent [31] introduced the Agent-Computer Interface, a file-system-based approach for resolving GitHub issues that allowed navigating and editing repository files. Our approach is based on single-script generation using a stateful IPython kernel, which eliminates file I/O overhead for this class of problems. CodeAct [28] promotes executable Python code as a unified action space, and finds that this approach still outperforms more restrictive formats. We apply this general approach to constraint modeling, where a general action space can solve specialized tasks using focused prompts. The iterative refinement that we use in CP-Agent is related to automated program repair. ChatRepair [30] proposed conversational repair, where one iteratively refines patches based on test failures. RepairAgent [3] considers repair as an autonomous task with tools for code search, patching, and testing. Our approach is similar in spirit to iterative code repair, where refinement is used to correct errors. Still, we apply it to generative tasks, translating natural language into constraint programs rather than repairing existing code.

The validity of an evaluation depends on benchmark quality. EvalPlus [13] showed that popular benchmarks have insufficient tests, while augmented test suites reveal previously undetected errors. Only recently, ClassEval [4] proposed a new benchmark for structural complexity through manual curation. We address problem ambiguity in the original CP-Bench problems and propose a fixed version of the benchmark that allows more rigorous testing of constraint modeling capabilities.

3 Agentic Python Coder

Our CP-Agent is based on a generic Python coding agent, equipped with a special prompt for constraint modeling. Our coding agent implements the basic idea of ReAct [32] (Reason and Act) in a way that is both minimal and capable. The architecture can be adapted to new problem domains through prompt engineering rather than code modification. By this separation, domain-agnostic agent capabilities remain distinct from task-specific knowledge. Our implementation is publicly available on GitHub and via PyPI for installation via “pip install agentic-python-coder” [20].

3.1 Architecture

Virtual execution space. Unlike software engineering agents, which navigate complex file trees, we do not maintain multiple files; instead, development is focused on a single file. We process development, refinement, and execution in a virtual space provided by the IPython kernel [16], which runs as a subprocess. The kernel provides a persistent state across multiple code executions. This guarantees that variables, functions, and imports remain accessible throughout a session and enables rapid hypothesis testing without file I/O overhead. The agent proceeds in rounds, and each round consists of code manipulation followed by immediate feedback via the Read–Eval–Print Loop (REPL). This design is well-suited for

developing constraint programming models and similar focused generation tasks.

The ReAct agent is implemented using LangGraph modules. LangGraph provides an important link between workflow management and durable execution, enabling the development of stateful agents. This even prevails when we consider the ReAct pattern [32] as interleaved reasoning and acting, which allows the agent to interact with the execution environment and adjust solutions through iterative cycles.

We will now explain the details of each component: the *ReAct* loop controller, which implements the reasoning-action cycle (using LangGraph [12]); three *tools* the agent can call; and a *project prompt* that provides domain-specific guidance. In each iteration of the ReAct loop, the agent reasons about the task, selects and executes tools, and observes results until a final solution is obtained and saved to a file (Figure 1).

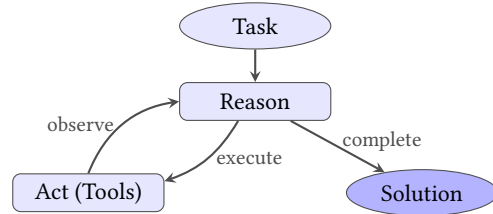


Figure 1: The ReAct (Reason and Act) framework employed by the agent.

Tools. The main tool is `python_exec(code: str)`, which runs Python code in the persistent kernel environment and returns `stdout/stderr` for solver feedback. Each call to this tool builds incrementally upon the previous calls. The function `save_code` transforms the solution into a file on disk, signaling completion. One could omit the `save_code` tool since file access is also possible via `python_exec(code: str)`. However, having a dedicated tool for code saving has the advantage of easier file-path handling and makes it clear to the agent that calling this tool marks the end of its session. Optionally, we give the agent access to a task management tool to keep track of tasks in the form of a “to-do list.” The tool call has the format `todo_write(todos: List[Dict[str, Any]])`, where `todos` is the task list, and each entry specifies task attributes. By construction, each task must contain `id`, `content`, `status`, and `priority` fields, and only one task can be marked as “in_progress” at any time.

Dynamic dependency injection. We implement a custom kernel manager, which wraps the standard IPython kernel launch. Instead of having the agent run `pip install` commands or rely on prebuilt container images, we inject packages at startup. Using the `uv` package manager [2], we split package installation into an ephemeral environment that runs at kernel startup. The kernel resolves dependencies based on session configuration; each session runs in a clean environment with only the specified packages.

LLM integration. The LLM that drives the agent is provided via the OpenRouter API. This allows us to use models from different model-makers through a uniform API and configuration, and with

a single API key. We record all the agent’s steps and tool calls in log files. These logs provide valuable insight into the agent’s problem-solving process and resource consumption.

Prompt architecture. The agent operates with the following three-fold prompt hierarchy: the *system prompt* (approximately 100 lines) specifies how the agent can be used to reason about code and execute iterative development cycles, defining general coding behavior and tool usage patterns without task-specific specialization. The *project prompt* combines domain-specific knowledge (such as constraint modeling) with examples and best practices relevant to this particular problem class. The concrete task to solve is in the *task prompt*, typically read from a `task.md` file in the working directory or provided directly via a command line parameter. The same agent infrastructure can tackle diverse problem domains by simply replacing the project prompt.

4 CP-Agent

For constraint programming, the generic coding agent is equipped with a domain-specific project prompt that instructs it to use CPMpy as the modeling framework. No specialized constraint programming logic is embedded in the architecture. The agent can create and test simplified problem instances and verify the constraint model on example data. The agent can even implement custom validation logic by refining models based on solver feedback. This flexible workflow, together with the persistent execution environment, provides an excellent environment for discovering and correcting modeling errors that would escape detection in single-pass generation approaches.

4.1 CPMpy

We use CPMpy [7] as the constraint programming foundation primarily due to its Python integration. This library is based on NumPy for efficient handling of decision variables as arrays, making it well-suited for integration with the broader Python ecosystem. Its solver-agnostic design allows a more refined selection of backends and hence enables modeling problems once and solving them using various solvers, including CP, MIP, SMT, and SAT. This design is in line with findings from Michailidis et al. [14] that LLMs perform better with Python-based frameworks than with domain-specific languages, likely due to the billions of lines of Python code in their training data.

4.2 Project Prompt

The project prompt (`cpmpy.md`), under 50 lines, provides succinct guidance for constraint modeling. The complete prompt is available online at the GitHub repository [20]. The prompt contains the following core rules: use CPMpy’s declarative modeling rather than implementing search algorithms, output only valid JSON using `json.dumps()`, always import the `json` Python module when outputting JSON, check the exact required output format, and manually test solutions to ensure they satisfy the problem requirements. A basic code template demonstrates the standard structure, and brief sections cover three essential CPMpy constraints (`AllDifferent`, `sum`, `Circuit`) and optimization via `model.minimize()` and `model.maximize()`.

This minimal prompt is based on an ablation study. We initially experimented with a substantially longer prompt of approximately 800 lines, containing detailed modeling procedures, extensive problem-type catalogs, and debugging guidance. The shorter version performs on par with, or better than, the longer version across all performance metrics. This indicates that for modern LLMs with constraint-programming knowledge in their training data, concise guidance is sufficient; excessive detail may distract and consume input tokens without providing benefit.

5 Benchmark Problems

The CP-Bench collection [14] provides an excellent benchmark for testing automated constraint modeling capabilities. This collection comprises 101 constraint programming problems from four established sources: CSPLib [6], CPMpy examples [7], models by Kjellerstrand [11], and problems from the APLAI course at KU Leuven. The collection covers diverse combinatorial problem types, including scheduling, assignment, sequencing, graph problems, and combinatorial design. With 30 optimization and 71 decision problems, the collection provides a balanced coverage of constraint programming challenges.

Each problem in CP-Bench consists of a natural-language description in Markdown format, input data, required output variables, and a ground-truth model for verification. Solutions must output results in JSON format using the specified variable names. The ground-truth models enable semantic validation through constraint satisfaction checking rather than syntactic correctness or execution success. The ground-truth models are withheld from the agent and used solely for post-hoc validation.

5.1 Fixed Workflow Results

Michailidis et al. [14] evaluated fixed workflow approaches on CP-Bench, testing progressively detailed prompts including basic instructions, modeling guidelines, and API documentation, as well as retrieval-augmented in-context learning. Their results indicate that combining repeated sampling with self-verification using GPT-4-turbo achieves the highest accuracy of 70%. A general limit of these fixed workflow approaches is that they do not support testing hypotheses, debugging errors, or verifying partial solutions.

5.2 Benchmark Corrections and Clarifications

During testing, we encountered issues with some CP-Bench problems. The CP-Bench authors acknowledged these issues independently¹, including ground-truth models with symmetry-breaking constraints that reject valid solutions, unclear output format requirements, and ambiguities in problem statements. Without a clear problem specification, the modeling ability cannot be evaluated separately from format guessing. We therefore updated the CP-Bench problems to address these issues.

Our benchmark updates fall into three categories.

First, we clarified 31 ambiguous problem statements. For instance, Problem 080 (TSP) required specifying whether Euclidean distances should be rounded or truncated, which can lead to different distance matrices and thus different optimal solutions. For Problem 013 (magic square), we corrected a misleading description that stated

¹personal communication, August 2025

the goal is to find a “magic square” when the problem actually requires a “magic sequence.” Problem 060 (crossword) provided a visual grid and word list, but required explicit specification of slot lengths and intersection constraints to be solvable.

Second, we added structured JSON schemas to all problems to clarify the required output format. We recommend this practice for any future extensions of the benchmark collection, as it eliminates trivial but annoying discrepancies, such as whether array indices are supposed to start with 0 or 1.

Third, we corrected errors in 19 ground-truth models: we removed symmetry-breaking constraints from 10 and fixed logic errors in 9 others.

While these updates do not make the problems easier, they remove confounding factors that obscure actual modeling capability. The underlying combinatorial structure and search space complexity remain the same; no changes hint at constraint formulations or modeling strategies. The success rates reported in the original CP-Bench paper were obtained without using our clarifications. A direct comparison between our results and theirs is not possible, as the fixed workflow can also benefit from the clarifications. The clarified benchmark is available online [22].

6 Experiments

6.1 Experimental Setup

We ran all our experiments on all 101 CP-Bench problems three times, with CP-Agent implemented as agentic-python-coder (version 2.2.1) using Claude Sonnet 4.5 (version 20250929) as the language model for our main experiment. The models used in the model generalization and the task management experiments are described below. For each problem, the agent produced a solution file containing the generated CPMpy model, and we recorded a conversation log with the complete agent interaction. We tested all solutions against the ground-truth CPMpy models. For each satisfaction problem, we tested whether the solution satisfies all ground-truth constraints. For optimization problems, we compared the objective value of the generated solution with an optimal pre-computed ground-truth value to confirm optimality. Note that a problem may have several correct and optimal solutions, hence the ground-truth model must not contain symmetry-breaking constraints. We always report results averaged across three runs to account for randomness.

The agentic-python-coder, the problem specifications and ground-truth models to reproduce our experiments, and the generated solutions and execution logs, are available online [22]. The data is encrypted (password Ckc1ADsKXRfmzD2) to avoid training data contamination.

6.2 Results

Our experimental results show that CP-Agent achieves 100% accuracy, successfully solving all 101 CP-Bench problems across all three independent runs. We validated all generated solutions against the ground-truth models and confirmed that all solutions were correct and optimal.

Our results are summarized in Table 1. The number of python_exec calls varies and depends on the structure of the problem, ranging from 1 to 28, with simpler problems requiring fewer

iterations and complex scheduling or optimization problems requiring more exploration. Problem 014 (crossfig) stands out with an unusually large number of Python executions (28 average), followed by 009 (basketball) with 24 executions, reflecting their combinatorial complexity.

The table also shows the token consumption. On average we have 37k input and 3k output tokens per problem. As expected, Problem 014 (crossfig), the problem with the largest number of exec calls has also the largest token consumption (737k input tokens).

The specifics of execution times differ from problem to problem, ranging from 20 seconds to 542 seconds, with an average of 52 seconds across the three runs. The average run time per problem in each of the three runs is 48, 57, and 52 seconds, respectively, which demonstrates the stochastic nature of LLM-based systems, but can also reflect variety in API response time and network speed.

We present representative runs on two problems to illustrate how the agent approaches constraint modeling, with different problem-solving patterns.

6.3 Problem 061: Cryptarithmic Puzzle

The problem is to assign letters with unique digits (0–9) such that the given 20-digit addition equation holds. The log reveals how the agent adapts when an initial approach proves unworkable.

In the first four executions, the agent introduces integer variables (one per letter), an AllDifferent constraint, mappings from variables to their positions in the three numbers, and constraints forcing leading digits to be non-zero. This completes the description of the problem structure.

The fifth execution attempts to treat each 20-digit number as a single large integer variable. The agent constructs expressions like `num1 = letter[0]*10**19 + letter[1]*10**18 + ...` for all three numbers and adds the constraint `num1 + num2 == result`. This approach fails because variable domains of size 10^{20} exceed the solver’s integer representation capacity, causing a `TypeError: __init__(): incompatible constructor arguments`.

With this feedback, the agent redesigns the model, introducing binary carry variables and models addition column by column, thereby mimicking manual arithmetic:

```
carries = intvar(0, 1, shape=21)
for pos in range(20):
    digit1 = letter_to_var[num1_chars[19-pos]]
    digit2 = letter_to_var[num2_chars[19-pos]]
    result_digit = letter_to_var[result_chars[19-pos]]
    carry_in = carries[pos]
    carry_out = carries[pos+1]
    model += (digit1 + digit2 + carry_in
              == result_digit + 10 * carry_out)
```

This way, all intermediate values are kept within standard integer bounds, enabling the solver to determine the correct digit assignment.

Table 1: Tool usage, token consumption, and execution time for all 101 CP-Bench problems (averaged over three independent runs). Column ex shows the number of python_exec calls (iterative refinement cycles). Columns in/out show input/output tokens in thousands. Column time shows execution time in seconds (rounded).

Problem	ex	in	out	time	Problem	ex	in	out	time	Problem	ex	in	out	time
001_car_seq	2.7	24	2	43	035_assignment	2.7	22	2	49	069_jobshop	3.3	29	3	52
002_template	2	21	2	52	036_autoref	4.3	31	2	46	070_knapsack	1	13	1	22
003_autocorr	3.3	26	2	44	037_bales_of_h	4	33	3	66	071_mario	7.3	86	6	110
004_golomb	2.3	22	2	42	038_bananas	1.7	18	2	39	072_minesweeper	5	37	3	57
005_all_inter	1.7	19	2	32	039_best_host	1.7	18	2	33	073_n_puzzle	11	106	6	119
006_vessel	2.3	26	3	51	040_big_bang2	1.7	21	2	39	074_packing_re	8.3	75	5	83
007_perfect_sq	4	30	2	44	041_bin_packin	1	13	1	23	075_resource_c	5.7	68	4	78
008_social_gol	7.7	54	3	61	042_birthday_c	1	14	1	26	076_room_assig	2	21	2	38
009_basketball	24	256	12	245	043_bowls_and	1.7	16	1	30	077_send_more	1	14	1	25
010_nonogram	8	90	6	110	044_broken_wei	2	19	2	40	078_set_game	3.3	34	3	60
011_progressiv	7.3	73	5	90	045_building_b	2.3	20	2	34	079_sudoku	2	23	2	39
012_schurs	2.7	23	2	36	046_cabbling	1.3	17	2	36	080_tsp	3.3	24	2	40
013_magic_sq	1	15	1	22	047_calvin_puz	2.7	24	2	49	081_who_killed	2	21	2	42
014_crossfig	28	737	33	542	048_candies	1.3	16	2	28	082_wolf_goat	3.3	28	3	58
015_langford	1.7	19	2	36	049_capital_bu	1	14	1	25	083_zebra	3.7	38	4	59
016_sports	8.3	75	5	99	050_chess_set	6.3	79	3	59	084_bank_card	1	13	1	23
017_bibd	2.7	24	2	37	051_circling_s	2.3	25	3	47	085_climbing_s	1.3	16	1	31
018_word_des	6.3	47	3	80	052_circular_t	3.7	29	2	47	086_color_simp	2	18	1	30
019_steiner	3.7	29	2	41	053_clock_trip	1	14	1	26	087_exodus	5	45	4	71
020_num_part	2.7	21	2	60	054_cmo_2012	3	25	3	49	088_farmer_and	2.3	19	2	34
021_diamond	4.3	41	4	77	055_coin3_appl	2	18	2	34	089_five_floor	1	14	1	24
022_graceful	1	16	1	26	056_coins_grid	4.3	38	2	47	090_grocery	4.3	31	2	51
023_n_queens	2	20	2	42	057_contractin	1	14	1	25	091_guards_and	2	20	2	37
024_costas	2	20	1	30	058_covering_o	2	24	2	42	092_hardy_1729	1.7	15	1	28
025_hadamard	4.3	31	2	46	059_crew	2.3	36	4	62	093_kidney_exc	3.3	27	2	44
026_abbots_puz	1	13	1	23	060_crossword	4	40	3	55	094_magic_sq	1.3	15	1	28
027_added_corn	2	20	2	37	061_crypta	4.7	67	4	74	095_maximal_in	1.7	19	2	36
028_ages_of_th	4	27	2	53	062_eighteen_h	1	12	1	21	096_money_chan	1	13	1	24
029_age_changi	4.7	38	3	57	063_facility_l	3	29	3	57	097_movie_sche	2.7	23	2	41
030_allergy	2	21	2	44	064_fifty_puzz	1.3	15	1	23	098_people_in	1	15	2	36
031_among	2	17	1	42	065_three_coin	1	15	1	27	099_subsets_10	1	14	1	27
032_appointmen	1	14	1	36	066_three_sum	1	13	1	20	100_subset_sum	2.7	21	1	37
033_archery_pu	2.3	18	1	49	067_twelve_pac	1.3	14	1	25	101_thick_as_t	2.7	26	3	54
034_arch_frien	2	24	3	56	068_bus_schedu	1	14	1	27					

6.4 Problem 082: Wolf, Goat, and Cabbage

The above example demonstrated error recovery; this well-known river-crossing puzzle illustrates the agent’s behavior when proceeding without failures.

The first step is to establish state variables: `wolf_pos[t]`, `goat_pos[t]`, `cabbage_pos[t]`, and `boat_pos[t]`, which track locations (shore 0 or 1) across an 8-step timeline. The second step of the construction fixes boundary conditions—all entities start at shore 0 and end at shore 1. The third step encodes the movement mechanics: the boat crosses at each step, the farmer transports at most one item per trip, and item movements coincide with boat movements. The fourth step adds safety constraints:

```
for i in range(steps):
    model += ~((goat_pos[i] == cabbage_pos[i])
```

```
    & (goat_pos[i] != boat_pos[i]))
    model += ~((wolf_pos[i] == goat_pos[i])
    & (wolf_pos[i] != boat_pos[i]))
```

These constraints ensure that the goat and cabbage (or wolf and goat) do not share a shore without the farmer present. The systematic layering—state, boundaries, movement, and safety—naturally reflects the construction process. Each phase is traversed in order to verify correctness before proceeding to the next. The execution completes successfully on the first attempt.

These two behaviors of the agent can be observed in different phases of problem solving: complete redesigns when facing fundamental obstacles and methodical incremental construction when the path is clear. The agent uses adaptive approaches rather than fixed workflows.

Table 2: Multi-model evaluation (3 runs each). Numbers indicate successful runs out of 3. Model success rates: Sonnet 100%, Opus/GPT-5 97%, Haiku 87%, DeepSeek 83%, Grok 80%.

Problem	Sonnet 4.5	Opus 4.5	Haiku 4.5	GPT-5.1	Grok	DeepSeek
001_car_seq	3	3	3	3	3	3
002_template	3	2	3	3	3	2
004_golomb	3	3	3	3	2	3
007_perfect_sq	3	3	3	3	3	3
009_basketball	3	3	1	2	0	0
010_nonogram	3	3	3	3	3	2
013_magic_sq	3	3	3	3	3	3
015_langford	3	3	1	3	2	3
020_num_part	3	3	3	3	3	3
037_bales_of_h	3	3	3	3	2	3

7 Model Generalization

To study whether the approach generalizes beyond Claude Sonnet 4.5, we evaluated six frontier models on ten representative problems, with three runs per model. The models come from four model makers: Anthropic (Sonnet 4.5, Opus 4.5, Haiku 4.5), OpenAI (GPT-5.1-codex-mini), xAI (Grok-code-fast-1), and DeepSeek (v3.2). Due to infrastructure issues during the experimental period, we did not test with Google’s Gemini models. We ran each problem with a 20-minute timeout, using a temperature of 0.0 for each model.

We give the results in Table 2. The tested models achieved a success rate of 80–100%. Sonnet 4.5 maintained perfect accuracy. Opus 4.5 and GPT-5.1 both reached 97%. Haiku 4.5, Grok, and DeepSeek achieved 80–87% accuracy; most failures occurred on Problem 009 (basketball), though sporadic failures also affected other problems. Basketball requires coordinating nine constraint categories, including mirroring schemes, home/away patterns, rival pairings, and date-specific requirements—the highest constraint density in this subset.

For some runs, the solver timed out after 20 minutes. Looking at execution logs, we see that these agents were not converging toward correct solutions, suggesting the timeout did not cut short otherwise successful attempts.

Our results show that the architecture generalizes across model families. This indicates that our agentic workflow generalizes from our default model, Sonnet. Opus, considered superior to Sonnet, showed slightly weaker performance, possibly because the prompt was optimized for Sonnet. Prompt optimization frameworks such as DSPy [10] could be used to automatically tune the project prompt for specific models, potentially narrowing the performance gap observed with other models.

8 Task Management Tool

Explicit task tracking for LLM agents—maintaining a mutable list of pending, in-progress, and completed tasks—was introduced by BabyAGI [15] and formalized in plan-and-solve prompting [27]. Our

implementation provides a tool that realizes this pattern. Our architecture requires only `todo_write`, not a corresponding `todo_read`: without history compaction, all previous tool calls remain in the conversation context, so the agent can always see its earlier task updates. To evaluate whether explicit task management benefits constraint modeling, we compare our method with and without this tool.

For this experiment, we use a smaller model (Claude Haiku 4.5) than our default (Claude Sonnet 4.5) to reveal potential differences. We consider the following 40 problems, stratified by difficulty: 20 easy problems (solved in 27 seconds or less by Sonnet 4.5) and 20 hard problems (requiring 58 seconds or more). We run each problem three times with `todo_write` disabled and three times with it enabled. The results are summarized in Table 3.

On easy problems, compared with the baseline, `todo_write` results in a significant increase in token consumption (191%, from 15K to 44K average) and execution time (111%, from 18s to 38s), while improving success by just one run. On hard problems, `todo_write` even reduced success from 52/60 to 45/60 while still adding overhead in terms of token usage and longer execution time.

A closer look at individual problems reveals a pattern. The task management tool provides structure for extended problem-solving, which helps three previously failing hard problems—crossfig, sports, and progressive. In contrast, it causes failures on three problems that worked without it. The task-tracking overhead appears to distract the agent from finding solutions in some cases.

9 MCP Server

The Model Context Protocol (MCP) is a standardized interface for connecting language models with external tools and data sources [8]. MCP was initially introduced by Anthropic in late 2024 [1] as a tool for agent-tool interaction, and has since been adopted by major AI platforms and integrated into development environments. In December 2025, MCP governance transferred to the Linux Foundation.

To make our framework accessible from other agentic systems, beyond our own ReAct agent, we define the same persistent IPython kernel system as an MCP server (`ipython_mcp`). This allows it to be integrated with any MCP-compatible client. The MCP server is equipped with `python_exec` for code execution, the same tool central to our standalone agent. It also comprises three tools for controlling the coding infrastructure: `python_reset` to clear state and install packages, `python_status` to query session information, and `python_interrupt` to halt long-running computations. This interactive mode also serves an educational purpose: users can observe how natural language constraints are formalized into CPMpy models, request explanations, and refine models based on solver feedback. Once developed, the generated code can be deployed to other contexts.

This approach complements the MCP-Solver [21], which provides specialized tools for constraint modeling (MiniZinc, PySAT, Z3, ASP). In contrast to the MCP-Solver, our new `ipython_mcp` server provides general-purpose Python execution, with domain adaptation driven by prompts rather than tool specialization.

Table 3: Task management ablation on 40 problems (Claude Haiku 4.5, 3 runs per condition). Columns: p = success (out of 3), tok = tokens (K, averaged), t = time (s, averaged). Bold highlights problems where success changed.

Easy Problem	no todo			todo			Hard Problem	no todo			todo		
	p	tok	t	p	tok	t		p	tok	t	p	tok	t
013_magic_sq	3	15	26	3	39	35	008_social_gol	3	36	36	3	122	74
022_graceful	3	38	32	3	49	42	009_basketball	3	752	303	0	181	119
026_abbots_puz	3	35	29	3	57	40	010_nonogram	3	47	48	3	112	67
041_bin_packin	3	10	13	3	52	48	011_progressive	1	201	111	2	730	197
042_birthday_c	3	19	20	3	52	40	014_crossfig	0	1986	617	2	1592	492
049_capital_bu	3	10	18	3	37	41	016_sports	0	112	70	1	712	228
053_clock_trip	2	17	18	3	87	54	018_word_des	3	41	40	3	160	90
057_contractin	3	16	18	3	38	35	021_diamond	3	69	63	3	130	77
062_eighteen_h	3	15	16	3	57	49	037_bales_of_h	3	40	40	3	149	81
064_fifty_puzz	3	10	12	3	35	29	050_chess_set	3	10	16	3	38	31
065_three_coin	3	15	16	3	70	60	059_crew	3	80	55	0	137	75
066_three_sum	3	9	13	3	31	26	061_crypta	3	43	31	2	297	127
067_twelve_pac	3	9	12	3	39	33	071_mario	3	243	145	2	291	135
068_bus_schedu	3	18	20	3	50	39	073_n_puzzle	3	203	95	3	273	122
070_knapsack	3	9	11	3	37	32	074_packing_re	3	22	25	3	134	81
077_send_more	3	11	13	3	41	37	075_resource_c	3	55	51	3	86	61
084_bank_card	3	10	16	3	26	24	078_set_game	3	24	24	3	105	70
089_five_floor	3	11	14	3	33	30	082_wolf_goat	3	62	60	0	286	156
096_money_chan	3	13	22	3	32	34	083_zebra	3	30	32	3	150	91
099_subsets_10	3	16	19	3	28	26	087_exodus	3	123	88	3	223	106
Easy (60 runs)	59			60			Hard (60 runs)	52			45		

10 Conclusion

Our experiments provide several insights into the design and evaluation of agentic systems for constraint modeling.

Our results show that agentic workflows with execution feedback can reliably automate constraint modeling from natural language.

Our main experiment demonstrates that a short prompt performs very well (even outperforms an 800-line earlier version). This suggests that, for foundation models with constraint-programming knowledge, the role of the prompt is to set high-level strategy and the output format rather than to provide exhaustive procedural instructions. This is in line with recent results on prompt compression showing that shorter, denser prompts can match or exceed verbose prompts [9]. In terms of procedural detail, excessive instruction might reduce the agent’s ability to solve problems effectively.

Our model generalization experiments show that our agentic architecture is robust: six models from four providers achieved success rates of 80–100%, with performance correlating with model capability.

Our experiment with the task management tool gives a mixed picture. While the tool reduces the overall success rate and significantly increases time and token usage, it was helpful for a few problems. Based on this outcome, we turn off the tool by default but keep it in the codebase for further testing, as there might be problem families where it shows a positive overall performance.

Finally, we reported on further integration possibilities of our system via the Model Context Protocol.

We believe that separating domain expertise from execution infrastructure, as realized in our system, can be beneficial in other scientific computing domains. In follow-up work, we used our system to transform natural language descriptions into logic programs [23], using the same architecture with only a domain-specific project prompt.

Acknowledgments

This research was supported by the Austrian Science Fund (FWF) projects 10.55776/P36420 and 10.55776/COE12.

References

- [1] Anthropic. 2024. Model Context Protocol. <https://modelcontextprotocol.io> Protocol specification for connecting language models to external systems.
- [2] Astral Software Inc. 2025. uv: An Extremely Fast Python Package and Project Manager. <https://github.com/astral-sh/uv> Documentation: <https://docs.astral.sh/uv/>.
- [3] Islem Bouzenia, Premkumar T. Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 2188–2200. doi:10.1109/ICSE55347.2025.00157
- [4] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating Large Language Models in Class-Level Code Generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 81, 13 pages. doi:10.1145/3597503.3639219
- [5] Eugene C. Freuder and Barry O’Sullivan. 2014. Grand challenges for constraint programming. *Constraints An Int. J.* 19, 2 (2014), 150–162. doi:10.1007/S10601-013-9155-1
- [6] Ian P. Gent and Toby Walsh. 1999. CSP_{LIB}: A Benchmark Library for Constraints. In *Principles and Practice of Constraint Programming - CP’99, 5th International Conference, Alexandria, Virginia, USA, October 11-14, 1999, Proceedings (Lecture*

- Notes in Computer Science*, Vol. 1713), Joxan Jaffar (Ed.). Springer, 480–481. doi:10.1007/978-3-540-48085-3_36
- [7] Tias Guns. 2025. Modeling and solving with CPMpy. <https://cpmpy.readthedocs.io/> CPMpy documentation.
 - [8] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. *CoRR* abs/2503.23278 (2025). arXiv:2503.23278 doi:10.48550/arXiv:2503.23278
 - [9] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*. 1658–1677. <https://aclanthology.org/2024.acl-long.91/>
 - [10] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *International Conference on Learning Representations (ICLR)*.
 - [11] Håkan Kjellerstrand. 2024. Constraint Programming Models Collection. <https://github.com/hakank/hakank> Collection of constraint programming models in various languages.
 - [12] LangChain. 2024. LangGraph: Build resilient language agents as graphs. <https://python.langchain.com/docs/langgraph> Python library for building stateful, multi-actor applications with LLMs.
 - [13] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html
 - [14] Kostis Michailidis, Dimos Tsouros, and Tias Guns. 2025. CP-Bench: Evaluating Large Language Models for Constraint Modelling. *CoRR* abs/2506.06052 (2025). arXiv:2506.06052 doi:10.48550/ARXIV.2506.06052
 - [15] Yohei Nakajima. 2023. BabyAGI: Task-Driven Autonomous Agent. <https://github.com/yoheinakajima/babyagi> GitHub repository, March 2023.
 - [16] Fernando Pérez and Brian E. Granger. 2007. IPython: A System for Interactive Scientific Computing. *Comput. Sci. Eng.* 9, 3 (2007), 21–29. doi:10.1109/MCSE.2007.53
 - [17] Rindranirina Ramamonjison, Timothy T. L. Yu, Raymond Li, Haley Li, Giuseppe Carenini, Bissan Ghaddar, Shiqi He, Mahdi Mostajabdaveh, Amin Banitalebi-Dehkordi, Zirui Zhou, and Yong Zhang. 2022. NL4Opt Competition: Formulating Optimization Problems Based on Their Natural Language Descriptions. In *NeurIPS 2022 Competition Track, November 28 - December 9, 2022, Online (Proceedings of Machine Learning Research, Vol. 220)*, Marco Ciccone, Gustavo Stolovitzky, and Jacob Albrecht (Eds.). PMLR, 189–203. <https://proceedings.mlr.press/v220/ramamonjison22a.html>
 - [18] Florian Régis, Elisabetta De Maria, and Alexandre Bonlarron. 2024. Combining Constraint Programming Reasoning with Large Language Model Predictions. In *30th International Conference on Principles and Practice of Constraint Programming (CP 2024) (LIPIcs, Vol. 307)*. 25:1–25:21. doi:10.4230/LIPIcs.CP.2024.25
 - [19] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html
 - [20] Stefan Szeider. 2025. Agentic Python Coder. <https://github.com/szeider/agentic-python-coder> GitHub repository.
 - [21] Stefan Szeider. 2025. Bridging Language Models and Symbolic Solvers via the Model Context Protocol. In *28th International Conference on Theory and Applications of Satisfiability Testing (SAT 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 341)*, Jeremias Berg and Jakob Nordström (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 30:1–30:12. <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.SAT.2025.30>
 - [22] Stefan Szeider. 2025. Supplementary Material for CP-Agent: Agentic Constraint Programming. doi:10.5281/zenodo.18034815 Zenodo.
 - [23] Stefan Szeider. 2026. ASP-Bench: From Natural Language to Logic Programs. In *2nd International Workshop on Neuro-Symbolic Software Engineering (NSE)*. To appear at ICSE 2026.
 - [24] Dimos Tsouros, Hélène Verhaeghe, Serdar Kadioğlu, and Tias Guns. 2023. Holy Grail 2.0: From Natural Language to Constraint Models. *CoRR* abs/2308.01589 (2023). arXiv:2308.01589 <https://arxiv.org/abs/2308.01589>
 - [25] Nemika Tyagi, Mihir Parmar, Mohith Kulkarni, Aswin RRV, Nisarg Patel, Mutsumi Nakamura, Arindam Mitra, and Chitta Baral. 2024. Step-by-Step Reasoning to Solve Grid Puzzles: Where do LLMs Falter?. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, 19898–19915. doi:10.18653/V1/2024.EMNLP-MAIN.1111
 - [26] Mark Wallace. 1996. Practical Applications of Constraint Programming. *Constraints An Int. J.* 1, 1/2 (1996), 139–168. doi:10.1007/BF00143881
 - [27] Lei Wang, Wanyu Xu, Yihui Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)*. Association for Computational Linguistics, 2609–2634. doi:10.18653/V1/2023.ACL-LONG.147
 - [28] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable Code Actions Elicit Better LLM Agents. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. <https://openreview.net/forum?id=iJ9BoXAfFa>
 - [29] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html
 - [30] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \$0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, Maria Christakis and Michael Pradel (Eds.). ACM, 819–831. doi:10.1145/3650212.3680323
 - [31] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globerson, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html
 - [32] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/forum?id=WE_vluYUL-X